Contents

1 Step 9 — Multi-Agent Reinforcement Learning: Experiment Report

Testbeds: four canonical 2×2 matrix games (Prisoner's Dilemma, Matching Pennies, Stag Hunt, Battle of the Sexes); Kuhn Poker and Leduc Hold'em (via the Step 07 exact stack); a native Goofspiel engine ($K = 3, 4$); two small cooperative environments (a one-step referential "CoopSignal" game and the Claus–Boutilier climbing game); and the memory-1 Iterated Prisoner's Dilemma. The poker games reuse the Step 02 Kuhn engine, the Step 03 Leduc engine, and Step 07's exact best-response / NashConv code wholesale.

PhD connection: the pivot from two-player zero-sum (Steps 2–8) into multi-agent RL. Three hooks: **LOLA** as *dynamic* opponent modeling (Contribution #1); **PSRO** as a population-level evaluation methodology (Contribution #3); and the **missing** $N > 2$ **minimax anchor** where two-player safety guarantees vanish (Contribution #2).

Scope of results: every number in this report is **measured from a real run** and read from the run artifacts under `implementation/step09/implementation/results/{smoke,scale}_results.json` and `implementation/step09/exploration/figures/*.json`. Wherever possible a simulated number is bracketed by an *exact* analytical reference (analytic Nash for the matrix games; Step 07's exact best response / NashConv for Kuhn, Leduc, and Goofspiel). Four Phase-4 predictions were **contradicted** by the runs; per the project workflow (§0.1) they are kept as stated and reconciled with what actually happened (§9).

How to read this report. Both parts follow the same arc: **what we test → how → results → conclusion. Part I (§1–§4)** shows *why* multi-agent learning is hard, on matrix games and Kuhn self-play. **Part II (§5–§8)** evaluates the structural fixes — PSRO, CTDE, communication, LOLA — against exact yardsticks. §9 reconciles the four contradicted predictions; §10–§12 cover trust, limitations, and directions; §13 lists reproduction commands.

---

2 PART I — WHY MULTI-AGENT LEARNING IS HARD

2.1 1. What this step is about

Single-agent RL assumes a **stationary** environment; multi-agent RL does not, because the other agents are *also learning*, so each agent's effective environment is a moving target (**non-stationarity**). Part I makes that failure visible with the cleanest possible controls — independent gradient learners on games whose Nash equilibria are known exactly — before Part II introduces the machinery that repairs it. All Part I numbers come from the seeded exploration scripts (`exploration/figures/*.json`); the learners use **exact gradients**, so the dynamics carry no sampling noise and "did it converge?" is unambiguous.

2.2 2. Experiment 1 — independent learners on the four matrix games

What we test. Do two independent gradient learners reach the analytic Nash equilibrium of each canonical 2×2 game, and if not, how do they fail?

How. Two exact-gradient learners, 4000 steps, learning rate 0.1, symmetric start $(0.5, 0.5)$, seed 0. "Converged" means the last 100 steps barely moved. Data: `exploration/figures/matrix_games_playground.jso`

Game	p_{final} (row)	q_{final} (col)	row payoff	converged?	analytic Nash
Prisoner's Dilemma	0.0025	0.0025	1.008	yes	(Defect, Defect), value 1
Matching Pennies	0.508	0.496	−0.0001	**no**	mixed $(\frac{1}{2}, \frac{1}{2})$, value 0

Game	p_{final} (row)	q_{final} (col)	row payoff	converged?	analytic Nash
Stag Hunt	0.0009	0.0009	2.997	yes	(Hare, Hare) risk-dominant, value 3
Battle of the Sexes	0.0029	0.0014	0.996	yes	(Football, Football), row 1 / col 2

(Here action index 0 is Cooperate/Stag/Opera; $p \to 0$ means the row learner plays the *second* action — Defect/Hare/Football.)

Results. Three of four converge to a genuine Nash. Prisoner's Dilemma collapses to mutual defection (dominant strategy). Stag Hunt settles on the **risk-dominant** (Hare, Hare) rather than the payoff-dominant (Stag, Stag) — the safe corner wins under gradient dynamics. Battle of the Sexes coordinates on one pure equilibrium. Matching Pennies **does not converge**: the learners hover near $(0.51, 0.50)$ at step 4000 but the run reports *not converged*, and §3 shows the trajectory is actively moving.

Conclusion. Independent learning is the control that fails *on purpose*: it works where a game has a dominant strategy or a reachable pure equilibrium, and breaks where the only equilibrium is mixed. That break is the case for everything in Part II.

2.3 3. Experiment 2 — the non-stationarity orbit (Matching Pennies vs PD)

What we test. In Matching Pennies, does the learners' distance from the mixed Nash shrink over time (convergence) or not (non-stationarity)?

How. Track the distance-to-Nash in successive time-windows; contrast with Prisoner's Dilemma's distance-to-(Defect,Defect). Steps 6000, lr 0.1, off-center start $(0.7, 0.3)$, seed 0. Data: `exploration/figures/nonstationarity_demo.json`.

Time-window	Matching Pennies radius	Prisoner's Dilemma radius
1	0.301	0.079
2	0.349	0.010
3	0.391	0.0058

Time-window	Matching Pennies radius	Prisoner's Dilemma radius
4	0.425	0.0041
5	0.453	0.0032
6	0.476	0.0026

Results. Prisoner's Dilemma's radius collapses to zero (convergence). Matching Pennies' radius **grows**, from 0.30 to 0.48 — the trajectory spirals *outward* toward the boundary rather than settling or holding a constant orbit.

Conclusion. Non-stationarity is structural: more compute traces a larger divergence, not convergence. (This contradicts the Phase-4 "constant-radius orbit" prediction; see §9.)

2.4 4. Experiment 3 — self-play on Kuhn: average vs last iterate

What we test. In fictitious-play self-play on a real imperfect-information game, does the *average* strategy converge to Nash even when the *last* iterate does not?

How. Fictitious play on Kuhn (each player best-responds to the opponent's running average via Step 07's exact best response), 200 iterations, measured every 5. Track the average-iterate NashConv and the last-iterate NashConv. Data: `exploration/figures/selfplay_vs_nash.json`.

Iteration	average-iterate NashConv	last-iterate NashConv
5	0.240	0.500
50	0.063	0.500
100	0.046	0.500
150	0.038	0.500
200	0.031	0.500

(The last-iterate value oscillates across $\{0.33, 0.50, 0.67, 0.83\}$ throughout — it never trends down.)

Results. The average-iterate exploitability falls monotonically from 0.24 to 0.031; the last-iterate value keeps oscillating in $[0.33, 0.83]$.

Conclusion. Self-play converges **in the average, not the last iterate** — the concrete motivation for averaging (CFR, fictitious play) and for PSRO's meta-Nash *mixture* over a population rather than trusting the latest policy (§5).

---

3 PART II — THE STRUCTURAL FIXES

3.1 5. Experiment 4 — PSRO across game families

What we test. Does PSRO (population + meta-Nash + best-response oracle) drive the meta-Nash mixture's exploitability down toward zero, and does that hold across game sizes?

How. PSRO with Step 07's **exact** best-response oracle. The opponent's meta-Nash mixture over behavioral policies is collapsed to a single realization-equivalent behavioral policy (Kuhn's theorem, perfect recall) so the exact BR engine applies. Exploitability = NashConv of the meta-mixture in the full game. Data: `results/scale_results.json` (`psro` block); RPS from `exploration/figures/psro_peek.json`.

Game	round 0	final (round)	verdict
Kuhn Poker	0.917	$\sim 2 \times 10^{-16}$ (6)	converges to machine zero
matrix (Matching Pennies)	2.0	0.0 (2)	converges
Rock–Paper–Scissors	2.0	0.017 (4)	converges; mixture $\to$ $(0.335, 0.336, 0.329)$
Leduc Hold'em	4.75	2.16 (19)	decreases, **» 0.5 target**
Goofspiel ($K = 3$)	1.33	0.0 (1)	converges
Goofspiel ($K = 4$)	1.50	1.71; oscillates 1.4–2.0	**does not settle**

Results. On the small games (Kuhn, matrix, RPS, Goofspiel $K = 3$) exploitability collapses to (near) zero within a handful of rounds — textbook double-oracle behavior. Leduc declines steadily ($4.75 \to 2.16$) but stays far above the 0.5 target after 20 rounds. Goofspiel $K = 4$ oscillates between $\sim$1.4 and $\sim$2.0 without settling.

Conclusion. PSRO is validated as the game-theory MARL bridge on the small games, using the same exact exploitability metric as Steps 2–8. Two results — Leduc's slow convergence and Goofspiel $K = 4$'s oscillation — did not match predictions and are reconciled in §9.

3.2 6. Experiment 5 — CTDE: critic variance and the climbing game

What we test. (a) Does a *centralized* critic have lower value-target variance than *independent* critics? (b) Does a centralized critic let cooperative learners escape a miscoordination trap and beat

independent learners?

How. (a) MADDPG on the one-step CoopSignal task, training a centralized $Q(s, \text{joint } a)$ and per-agent $Q_i(o_i)$ on the same data; compare final residual loss. (b) Independent learners, MADDPG, and MAPPO on the Claus–Boutilier climbing game (optimum 11 flanked by −30 penalties; safe attractor 5); compare greedy team reward. Data: `results/scale_results.json` (`coop` block).

(a) Critic variance (CoopSignal):

Critic	final residual (value loss)
centralized $Q(s, \text{joint } a)$	3.17×10^{-11}
independent $Q_i(o_i)$	0.0766

(b) Climbing game greedy reward:

Learner	reward	(optimum 11, safe 5)
independent learners	7.0	—
MADDPG	5.0	—
MAPPO	7.0	—

Results. (a) The centralized critic's residual is essentially zero (it conditions on the target and the joint action, so the reward is deterministic in its inputs), orders of magnitude below the independent critic's 0.077 — the CTDE variance-reduction claim, confirmed. (b) **No method reaches the optimum**; independent learners and MAPPO reach 7, and **MADDPG trails at** 5 (the safe attractor).

Conclusion. A centralized critic is a lower-variance teacher (confirmed), but that alone does **not** solve hard-exploration coordination — the −30 penalties deter the agents from ever trying the joint action that reaches 11. MADDPG underperforming IL specifically flags its discrete counterfactual-baseline actor for scrutiny (§9, §11).

3.3 7. Experiment 6 — learned communication (CommNet)

What we test. Does a learned message channel let a listener that cannot see the target exceed the $1/K$ guessing ceiling?

How. CommNet on CoopSignal ($K = 5$, guessing ceiling 0.2), trained with the channel ON and OFF; compare greedy team reward. Data: `results/scale_results.json` (`coop.communication`).

Channel	greedy reward
communication ON	0.795
communication OFF	0.204

Results. With the channel the reward is 0.795, far above the 0.2 ceiling; without it, 0.204, exactly at the ceiling.

Conclusion. Communication is learned end-to-end and does real work — the speaker learns to encode the target and the listener to decode it. (At the smoke config both were 0.24; the effect requires training to convergence — §9.)

3.4 8. Experiment 7 — LOLA on the Iterated Prisoner's Dilemma

What we test. Do learners that differentiate through the opponent's *next* learning step cooperate where naive learners defect?

How. Memory-1 IPD with an exact closed-form return; naive gradient learners vs LOLA learners (look-ahead via nested finite differences). Compare per-step discounted return (full cooperation ≈ 3, mutual defection ≈ 1). Data: `results/scale_results.json` (`lola`); exploration curve in `exploration/figures/lola_ipd_playground.json`.

Learners	per-step return
naive vs naive	1.04
LOLA vs LOLA	2.82

(The exploration run, with a larger look-ahead rate, reached LOLA returns of $\sim$2.67–2.93 vs naive $\sim$1.06.)

Results. Naive learning converges to mutual defection (1.04); LOLA reaches near-cooperation (2.82). A built-in check confirms the mechanism: with the look-ahead rate set to zero, LOLA's gradient equals the naive gradient exactly, so the cooperation comes from the second-order term.

Conclusion. LOLA reshapes the learning dynamics — *dynamic* opponent modeling — turning IPD defectors into cooperators. This is the moving-target complement to Step 7's static opponent read (Contribution #1).

3.5 9. Prediction reality reconciliation

Per WORKFLOW §0.1, contradicted Phase-4 predictions are kept and reconciled, not silently edited. Four gaps:

1. **Matching Pennies orbit (§3).** *Predicted:* a clean orbit at roughly constant radius around $(\frac{1}{2}, \frac{1}{2})$. *Measured:* the radius grew $0.30 \to 0.48$ (outward spiral); the implementation's softmax learner instead drifted to the corners (NashConv $\sim$1.8). *Reconciliation:* the non-convergence lesson is intact and sharper; the "energy-preserving orbit" mental model was the error, and the actual boundary-drift is a stronger argument for averaging/population methods. Not a bug — two different, valid gradient dynamics.

2. **PSRO on Leduc (§5).** *Predicted:* exploitability < 0.5 within 20 iterations. *Measured:* $4.75 \to 2.16$ over 20 rounds — declining but far above 0.5. *Reconciliation:* genuine slow convergence, not a bug — Kuhn hit machine zero in 6 rounds, but Leduc's tree is far larger and a 20-member *pure-strategy* population cannot closely approximate its mixed Nash. The target was optimistic; the downward trend is the correct behavior. Verified by the clean Kuhn/matrix/RPS convergence on the same code path.

3. **Goofspiel $K = 4$ (§5).** *Predicted:* non-increasing exploitability. *Measured:* $K = 3$ converged to 0; $K = 4$ oscillates 1.4–2.0. *Reconciliation:* the one unresolved anomaly. Documented, not fixed (per the chosen path). Prime suspects for a follow-up: the Goofspiel PSRO driver never de-duplicates best-response policies, and a pure-strategy population is likely too weak for the larger game's mixed meta-Nash. Flagged as an open code item — **not** presented as a validated PSRO result.

4. **Climbing game (§6).** *Predicted:* the centralized critic reaches the optimum 11 and beats independent learners. *Measured:* nobody reached 11; IL and MAPPO reached 7, MADDPG trailed at 5. *Reconciliation:* the separate critic-variance claim held (central residual $\sim 3 \times 10^{-11}$ vs 0.077), so the honest split is "CTDE lowers critic variance CTDE solves hard-exploration coordination." MADDPG below IL additionally flags its counterfactual-baseline actor. The takeaway survives in chastened form.

A cross-cutting methodological note: the two neural effects (critic variance §6a; communication §7) were **invisible at the fast smoke config** — comm ON = OFF = 0.24, critic losses near-equal (0.0897 vs 0.0927) — and appeared only at the scale config. The scale numbers are the ones the neural claims rest on.

3.6 10. Trustworthiness and sample adequacy

- **Game-theoretic results are bounded by exact references.** Matrix outcomes are checked against analytic Nash; Kuhn/Leduc/Goofspiel exploitability is Step 07's *exact* NashConv (not a simulation). PSRO's Kuhn result reaching $\sim 2 \times 10^{-16}$ is machine-precision zero, the strongest possible confirmation.
- **Matrix and PSRO learners are deterministic** (exact gradients / exact BR + fixed seeds), so those numbers are exactly reproducible.
- **Neural results are seeds-limited.** The coop/comm/LOLA results are reported at one primary seed (scale allows 3); they are *qualitative inequalities* (central < independent; comm ON > OFF; LOLA > naive), robust in direction but seed- and library-version-sensitive in magnitude. They should not be read as precise point estimates.
- **Not captured on this run:** the `validate.py` PASS/FAIL log and the experiment PNGs (only the JSON artifacts were saved). Listed as "to close" in `../figures/README.md`.

3.7 11. Limitations (ranked by how much they affect the conclusions)

1. **Goofspiel $K = 4$ oscillation (§5, §9.3)** — an unexplained PSRO anomaly; the Goofspiel PSRO driver's lack of BR de-duplication and its pure-strategy population are the suspects. Until resolved, only the $K = 3$ Goofspiel result should be relied on.
2. **MADDPG underperforms IL on the climbing game (§6, §9.4)** — points at the discrete counterfactual-baseline actor update; the MADDPG *reward* results are therefore not trustworthy, though its *critic-variance* result (a separate, clean measurement) is.
3. **Leduc PSRO does not reach the target (§5, §9.2)** — genuine but means the "PSRO scales" claim is demonstrated only qualitatively (a downward trend), not to a low exploitability, on the larger game.
4. **Single-seed neural results (§10)** — direction-robust, magnitude-uncertain.
5. **Toy scale throughout** — matrix games, one-step coop tasks, tiny poker; nothing here should be extrapolated to deep-RL MARL at scale. (The compute policy correctly kept these small; a GPU was irrelevant.)
6. **Missing run artifacts** — no `validate.py` log or PNGs captured; figures are generated post-hoc from JSON.

3.8 12. Conclusions and research directions

Conclusions. The step delivers its arc end-to-end: independent learning fails exactly where theory says it must (Matching Pennies), and the structural fixes repair specific pieces — PSRO drives

exploitability to (near) zero on small games with the same exact metric as the game-theory steps; self-play works in the average, not the last iterate; a centralized critic is a near-zero-variance teacher; learned communication clears the guessing ceiling; and LOLA turns IPD defectors into cooperators. The honest negatives are as valuable: PSRO hits a scaling wall on Leduc, a centralized critic does not by itself solve hard-exploration coordination, and one PSRO variant (Goofspiel $K = 4$) misbehaves.

Research directions (each tied to a measured effect):

- *Approximate oracles for PSRO scaling* — Leduc's 2.16-after-20-rounds (§5) motivates an RL/approximate best-response oracle and measuring the guarantee it costs.
- *Fix and re-validate the two flagged pieces* — Goofspiel $K = 4$ de-duplication + mixed-strategy population (§9.3); MADDPG's counterfactual baseline (§9.4).
- *Dynamic + static opponent modeling* — combine LOLA's look-ahead (§8) with Step 7's static read (Contribution #1).
- *Safety without a minimax anchor* — the $N > 2$ gap named in the summary (§2 there): can PSRO's meta-game supply a usable safety substitute where no single game value exists (Contribution #2)?

3.9 13. Reproduction

From `implementation/step09/implementation/` with the project `.venv` active:

```
# validation harness (PASS/FAIL/SKIP against the raw-step targets)
python validate.py

# full comparison runs -> results/{smoke,scale}_results.json
python tournament.py --config smoke
python tournament.py --config scale

# plots from a results JSON -> plots/*.png  (needs matplotlib)
python plotting.py --config scale
```

From `implementation/step09/exploration/` (Part I figures + JSON):

```
python matrix_games_playground.py
python nonstationarity_demo.py
python selfplay_vs_nash.py
python psro_peek.py
python lola_ipd_playground.py
```

Seeds are fixed in `config.py` (implementation) and each exploration script's `CONFIG`. The ma-

trix and PSRO results are exactly reproducible; the neural results are direction-stable across seeds. All results in this report were read from `results/{smoke,scale}_results.json` and `exploration/figures/*.json`.